ML: Classification

CPE 232: Data Models

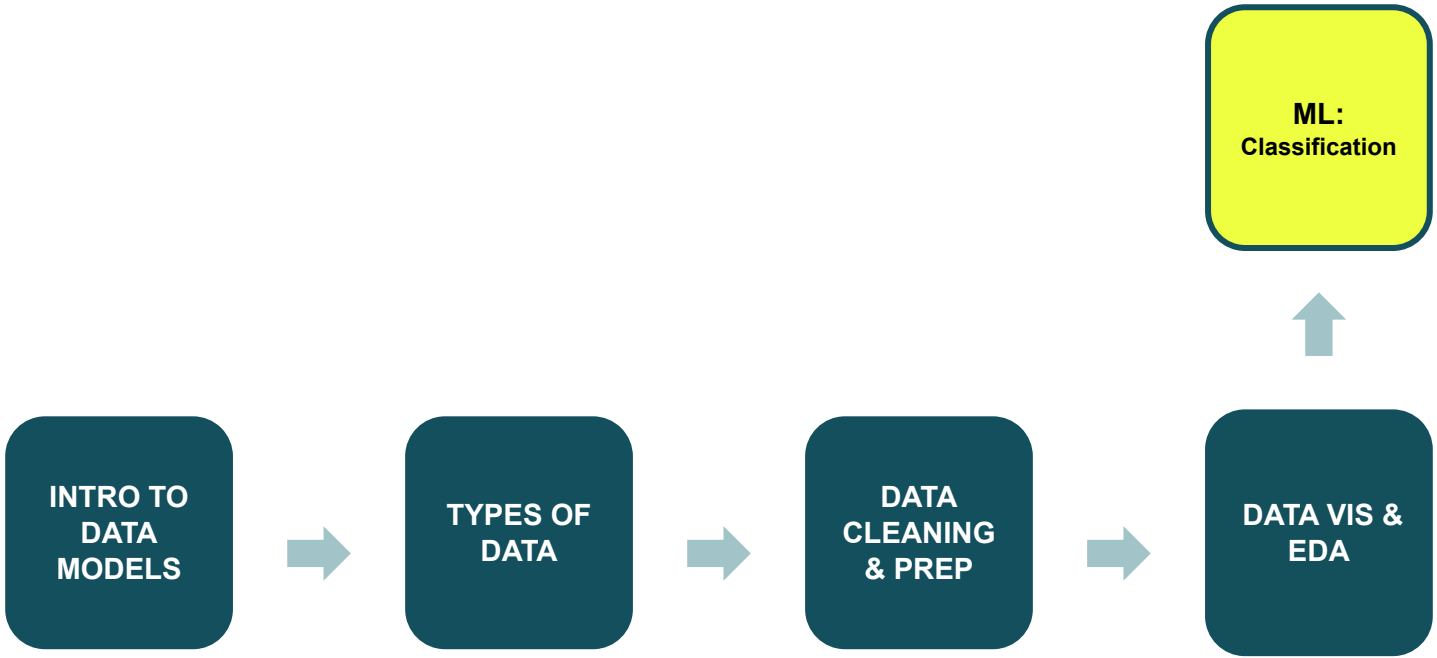
Dr. Sansiri Tarnpradab

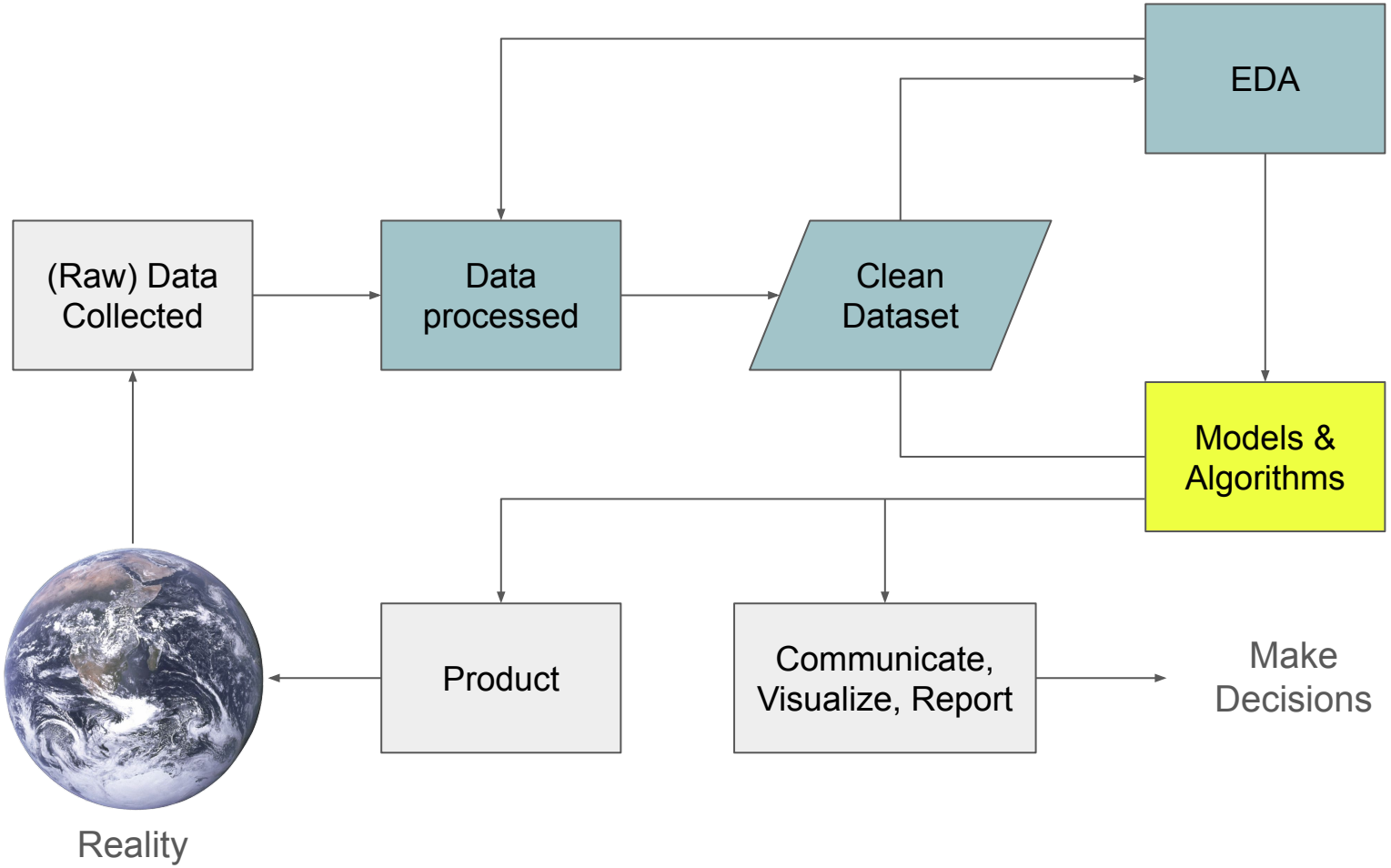
Department of Computer Engineering, KMUTT

Outline

- Introducing Model Concept
- Introducing Machine Learning
  - Algorithms
  - Types
- Classification Concept
- Decision Tree
- Activity

Review


INTRO TO DATA MODELS
TYPES OF DATA
DATA CLEANING & PREP
DATA VIS & EDA
ML:
Classification


EDA
(Raw) Data Collected
Data processed
Clean Dataset
Models & Algorithms
Product
Communicate, Visualize, Report
Make Decisions
Reality

Data Route in CPE

Sophomore (2nd)

Database

Data Models

Junior (3rd)

Big Data

Business Intelligence

Machine Learning

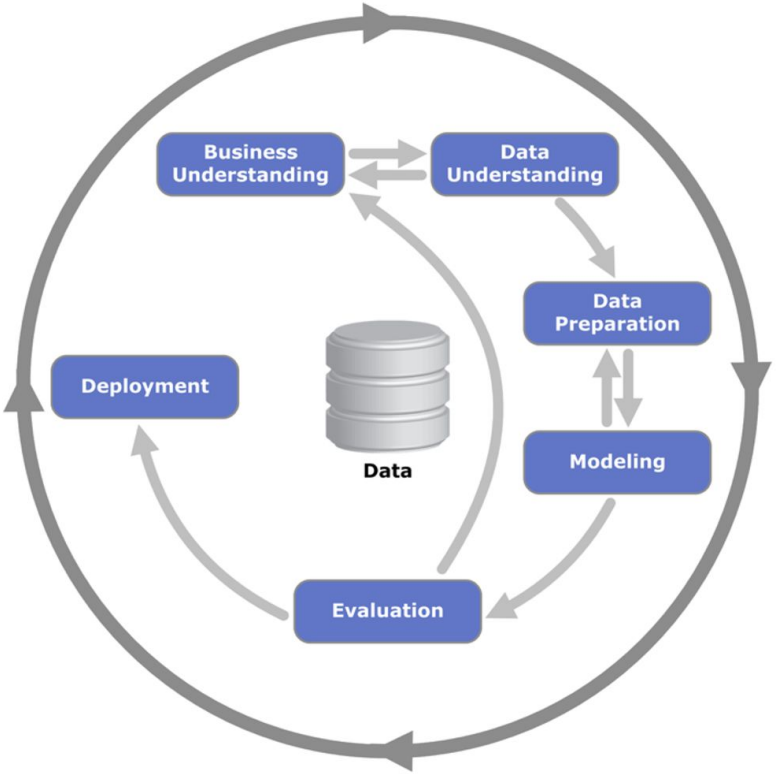
Text Analytics

Senior (4th)

Deep Learning

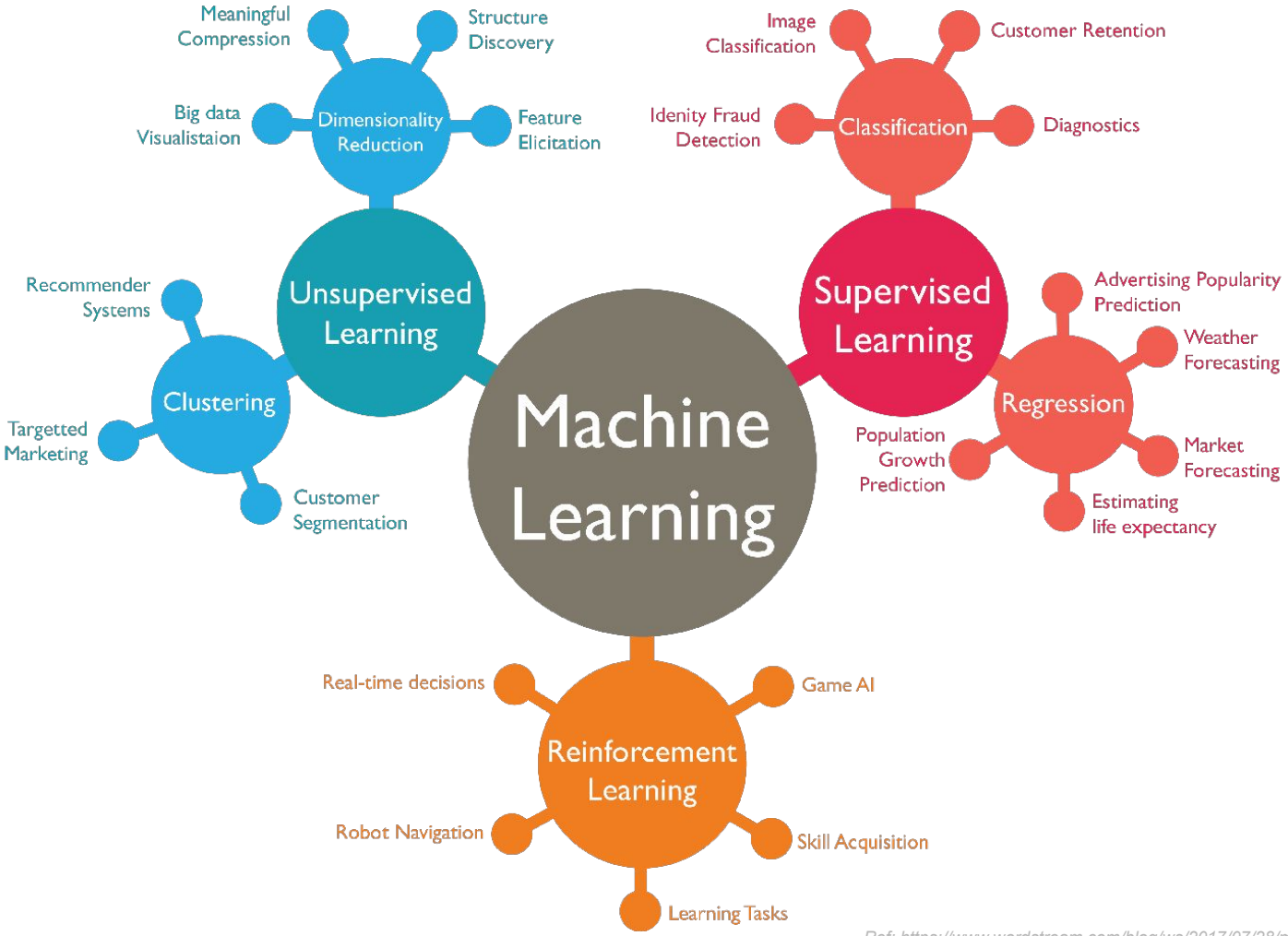
Data Mining

Recall this?


Business Understanding
Data Understanding
Data Preparation
Modeling
Evaluation
Deployment
Data

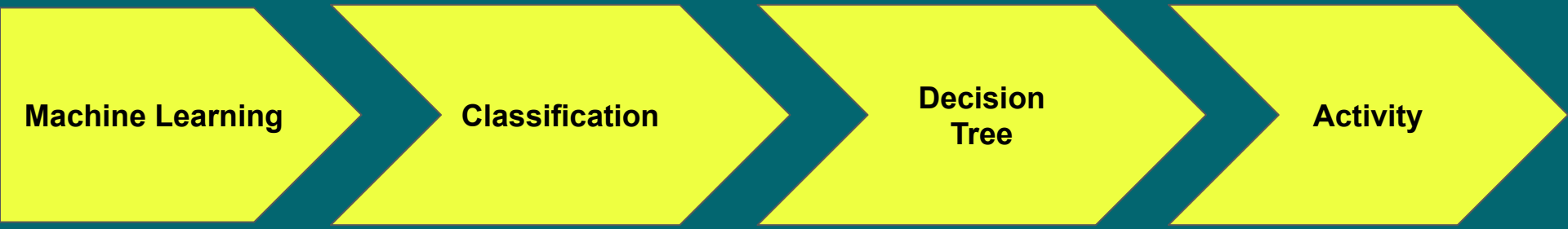
What is Model

- A simplified representation of reality created to serve a purpose
- Various professions have well-known model types
  - Architectural blueprints
  - Engineering prototype
  - SIR model for disease spreading (S: Susceptible, I: Infected, R: Recovered)
- Predictive Model
  - A formula for estimating an unknown value of interest (aka target)
  - Classification model
  - Regression model



Ref: https://www.wordstream.com/blog/ws/2017/07/28/machine-learning-applications

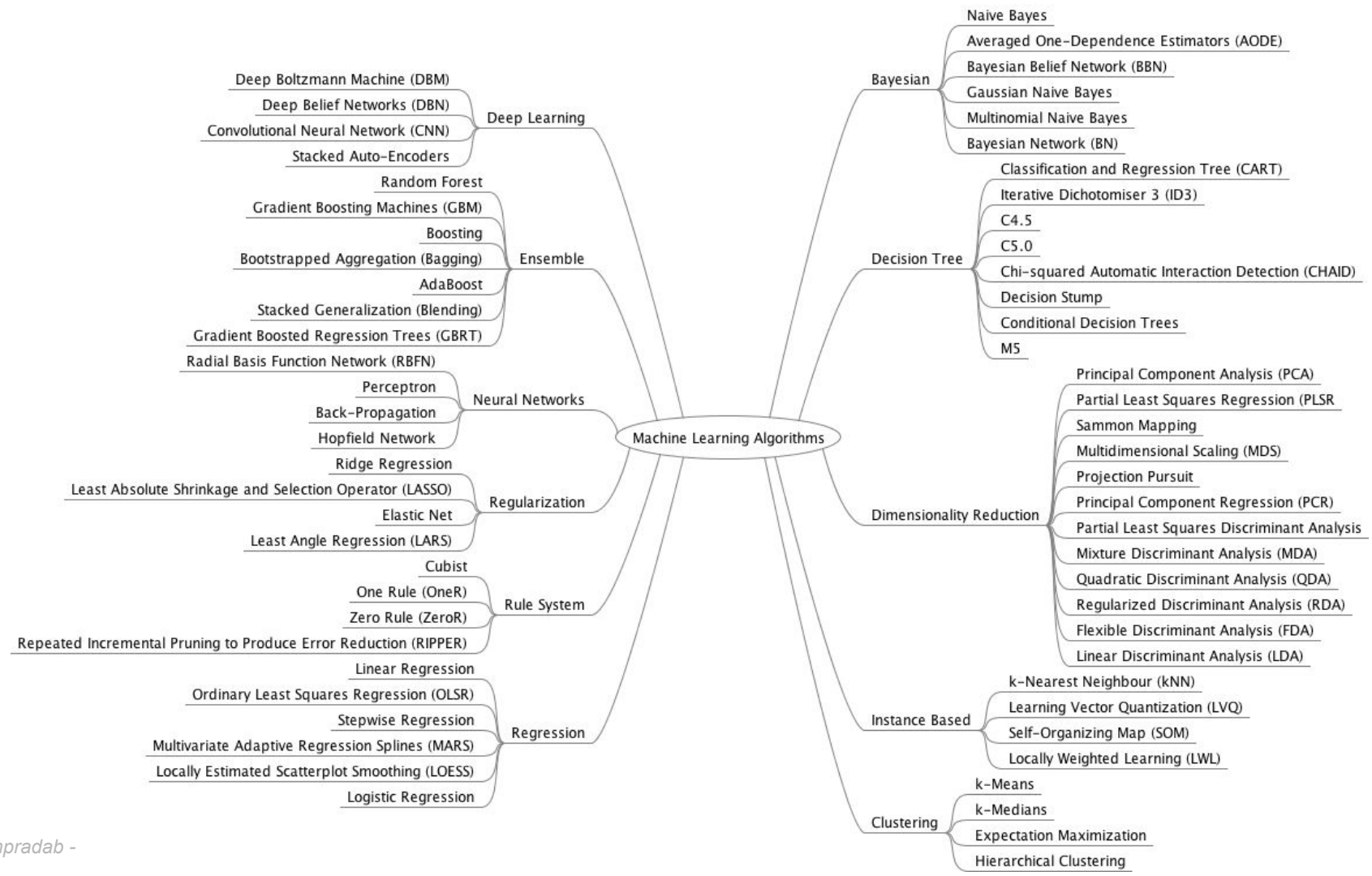
	Scenario	Techniques used
	Extracting the sound frequencies of a woman’s voice	
	Dividing the customers of a company according to their profitability	
	Computing the total income of a supermarket	
	Sorting a student database based on student names	
	Predicting the results of tossing a fair pair of dice	
	Monitoring the heart rate of a patient for abnormalities	
	Predicting the future stock market price using historical data	


Machine Learning
Classification
Decision Tree
Activity

What is Machine Learning

- Keywords
  - Machine
  - Learning

- The science of getting computers to learn from data without having to be explicitly programmed by humans.


Machine Learning Algorithms
Deep Learning
Deep Boltzmann Machine (DBM)
Deep Belief Networks (DBN)
Convolutional Neural Network (CNN)
Stacked Auto-Encoders
Ensemble
Random Forest
Gradient Boosting Machines (GBM)
Boosting
Bootstrapped Aggregation (Bagging)
AdaBoost
Stacked Generalization (Blending)
Gradient Boosted Regression Trees (GBRT)
Neural Networks
Radial Basis Function Network (RBFN)
Perceptron
Back-Propagation
Hopfield Network
Regularization
Ridge Regression
Least Absolute Shrinkage and Selection Operator (LASSO)
Elastic Net
Least Angle Regression (LARS)
Rule System
Cubist
One Rule (OneR)
Zero Rule (ZeroR)
Repeated Incremental Pruning to Produce Error Reduction (RIPPER)
Regression
Linear Regression
Ordinary Least Squares Regression (OLSR)
Stepwise Regression
Multivariate Adaptive Regression Splines (MARS)
Locally Estimated Scatterplot Smoothing (LOESS)
Logistic Regression
Bayesian
Naive Bayes
Averaged One-Dependence Estimators (AODE)
Bayesian Belief Network (BBN)
Gaussian Naive Bayes
Multinomial Naive Bayes
Bayesian Network (BN)
Decision Tree
Classification and Regression Tree (CART)
Iterative Dichotomiser 3 (ID3)
C4.5
C5.0
Chi-squared Automatic Interaction Detection (CHAID)
Decision Stump
Conditional Decision Trees
M5
Dimensionality Reduction
Principal Component Analysis (PCA)
Partial Least Squares Regression (PLSR
Sammon Mapping
Multidimensional Scaling (MDS)
Projection Pursuit
Principal Component Regression (PCR)
Partial Least Squares Discriminant Analysis
Mixture Discriminant Analysis (MDA)
Quadratic Discriminant Analysis (QDA)
Regularized Discriminant Analysis (RDA)
Flexible Discriminant Analysis (FDA)
Linear Discriminant Analysis (LDA)
Instance Based
k-Nearest Neighbour (kNN)
Learning Vector Quantization (LVQ)
Self-Organizing Map (SOM)
Locally Weighted Learning (LWL)
Clustering
k-Means
k-Medians
Expectation Maximization
Hierarchical Clustering

Basic Types of Machine Learning

Unseen Inputs

Labeled Data

Example inputs
Example outputs

Supervised Learning Algorithms

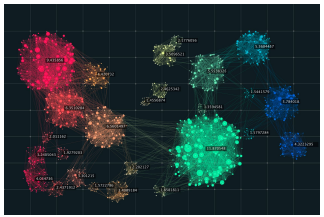
Prediction Model
(Classification/Regression)

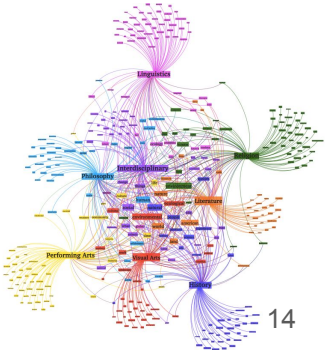
Predicted Outputs

Unlabeled Input Data

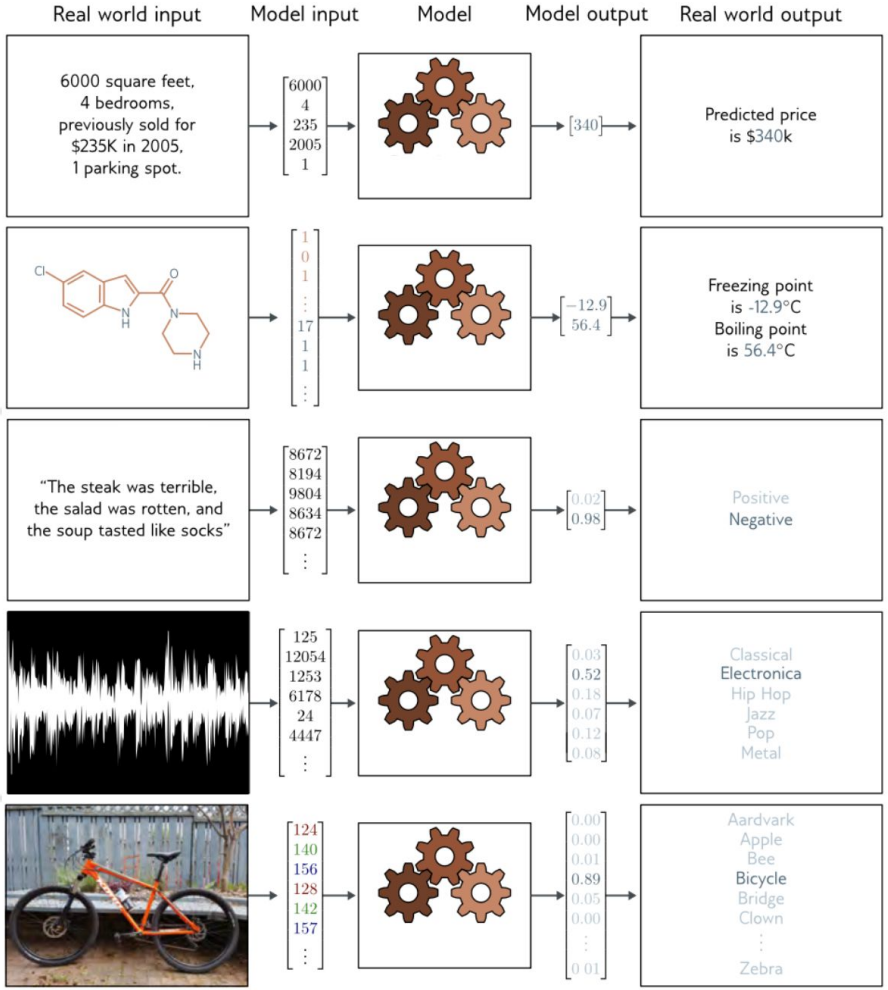
Unsupervised Learning Algorithms

Hidden/Unusual patterns
Alternative representations



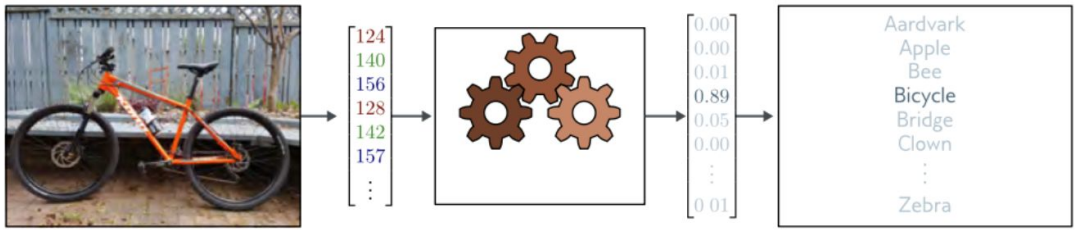


Refs:
Face Detection
Spam Detection
Topic Modeling
Social Networks

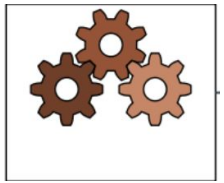


Ref: Prince, Understanding deep learning, 2023

Classification: In Practice

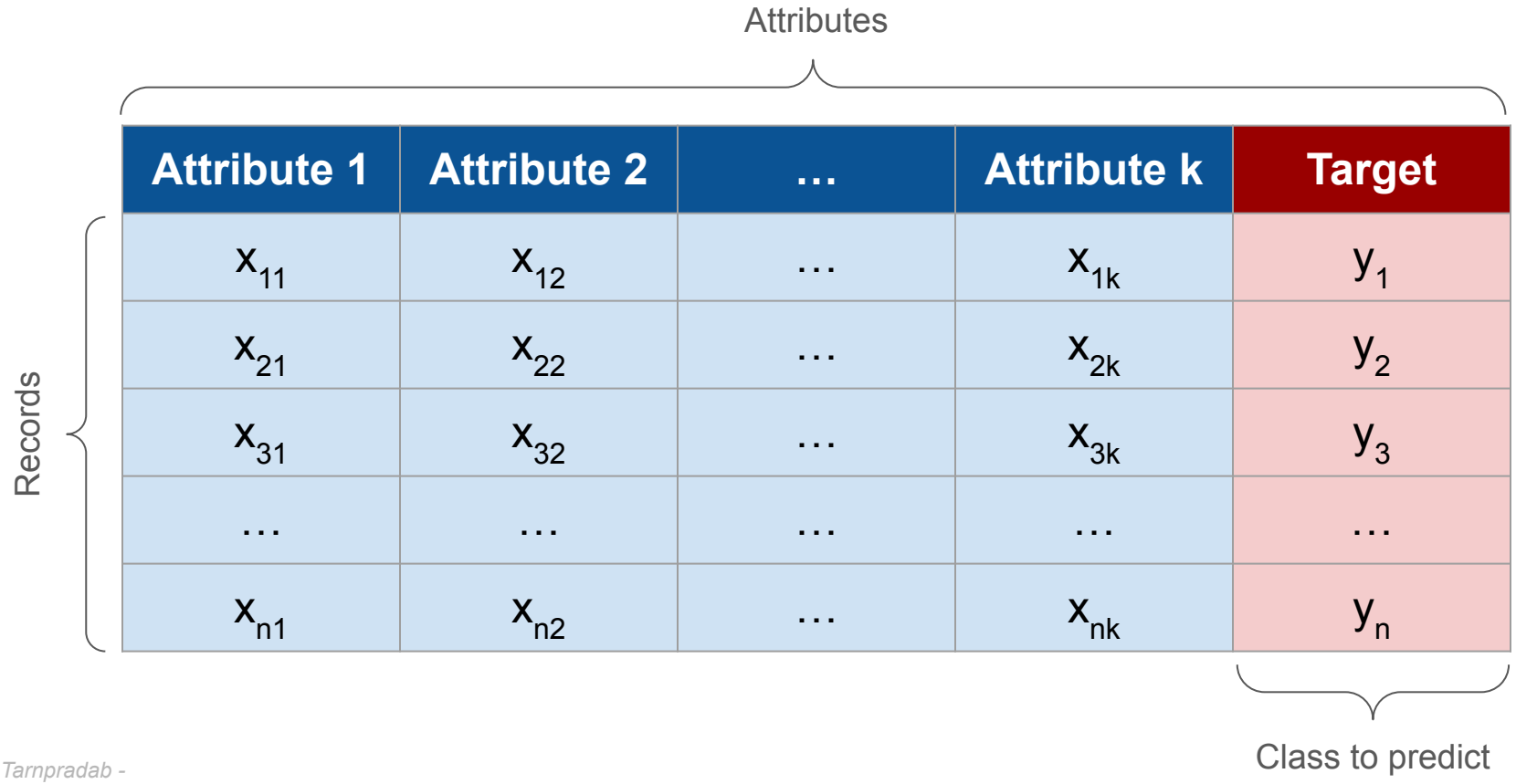


Number of tires
Size of tire
.
.
Color
Brand
Price
Chain
Reflectors
Lights
Brakes
Saddle
Handlebars
Frame



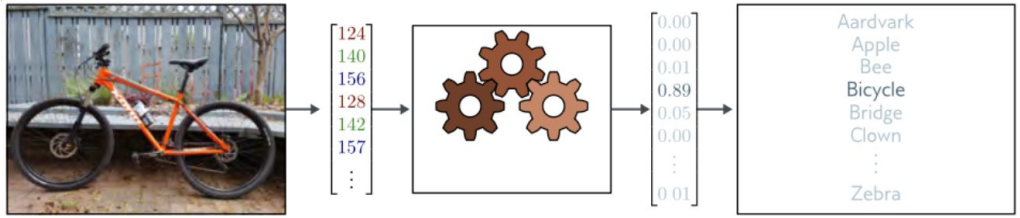
Bicycle
Or Else

Training Data

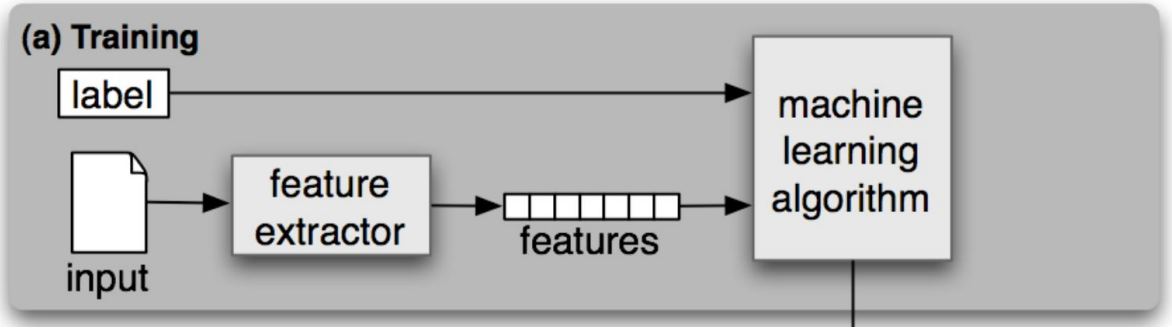


Classification: Steps

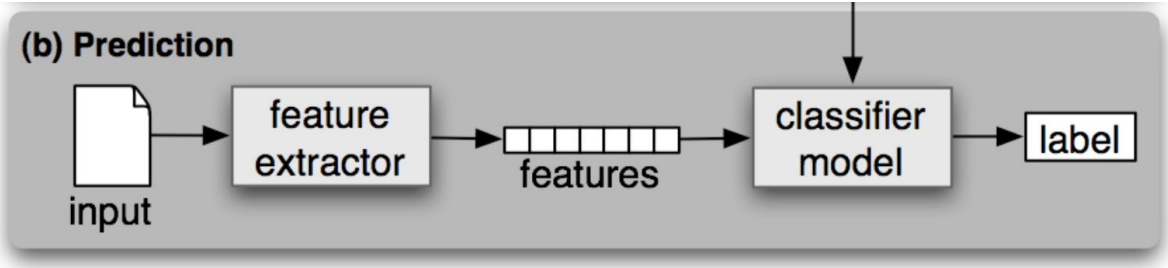
Image recognition
(Classification)



Feature Extraction	Choose relevant features that contribute most to the classification task
Data Splitting	Divide the dataset into training data and testing data
Model Training	Choose a classification algorithm and train it on the training data
Hyperparameter Tuning	Fine-tune the model by adjusting hyperparameters to optimize its performance
Model Evaluation	Evaluate the model performance on the testing data using evaluation metrics
Classification/Prediction	Make predictions on new unseen data

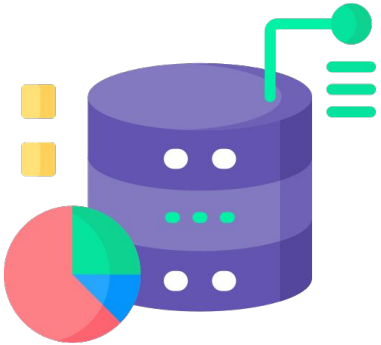


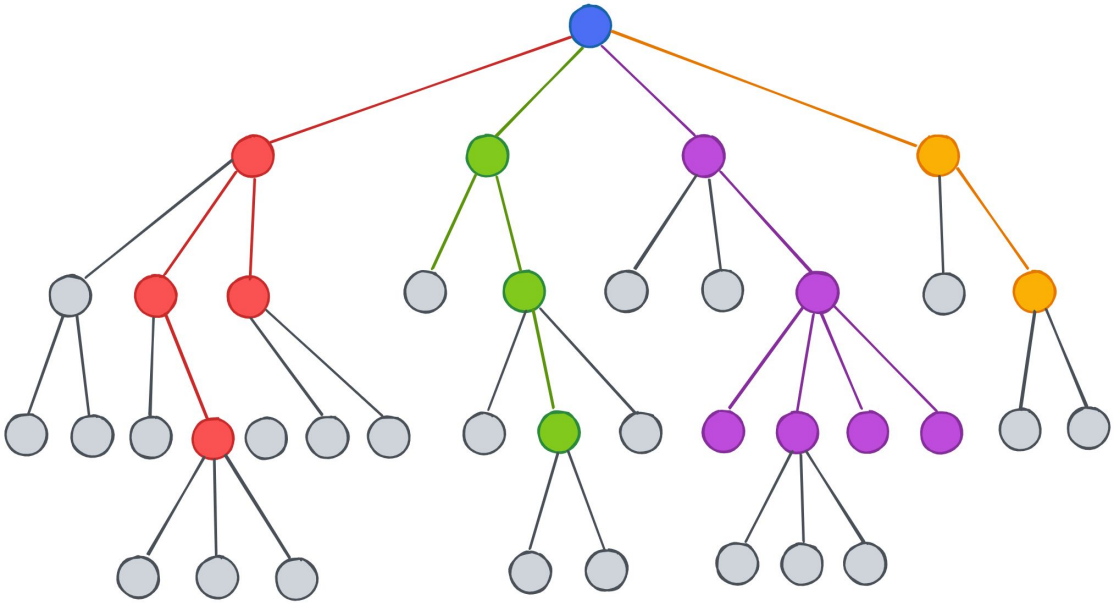
Feature Extraction	Choose relevant features that contribute most to the classification task
Data Splitting	Divide the dataset into training data and testing data
Model Training	Choose a classification algorithm and train it on the training data
Hyperparameter Tuning	Fine-tune the model by adjusting hyperparameters to optimize its performance
Model Evaluation	Evaluate the model performance on the testing data using evaluation metrics
Classification/Prediction	Make predictions on new unseen data



Feature Extraction	Choose relevant features that contribute most to the classification task
Data Splitting	Divide the dataset into training data and testing data
Model Training	Choose a classification algorithm and train it on the training data
Hyperparameter Tuning	Fine-tune the model by adjusting hyperparameters to optimize its performance
Model Evaluation	Evaluate the model performance on the testing data using evaluation metrics
Classification/Prediction	Make predictions on new unseen data

Example: Decision Tree

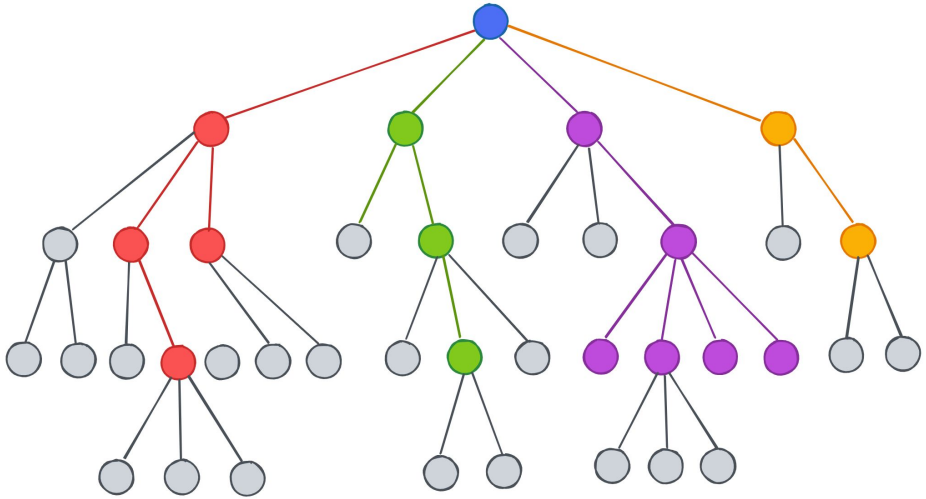




Refs:
Database Icon
Decision Tree

Decision Tree: Key Elements

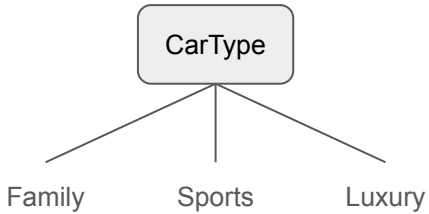
- Tree Induction = Training
- Best Split
- Node Impurity
- Entropy
- Information Gain

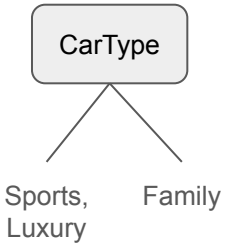


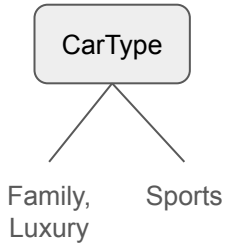
Ref:
Decision Tree

Determine The Best Split

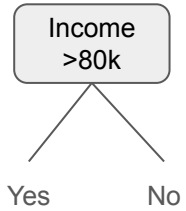
Ordinal Attributes

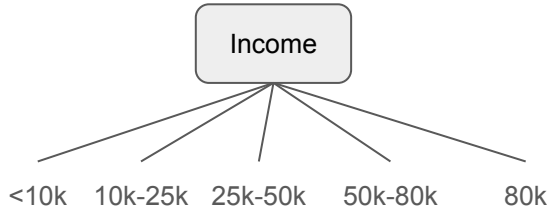




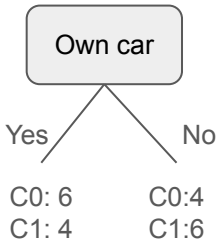


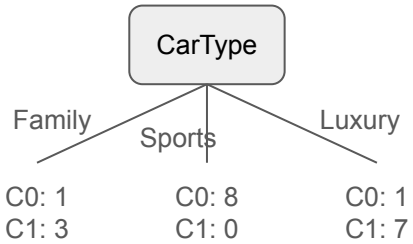
Continuous Attributes

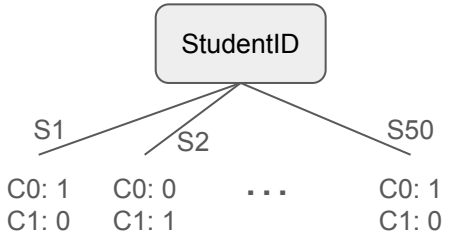




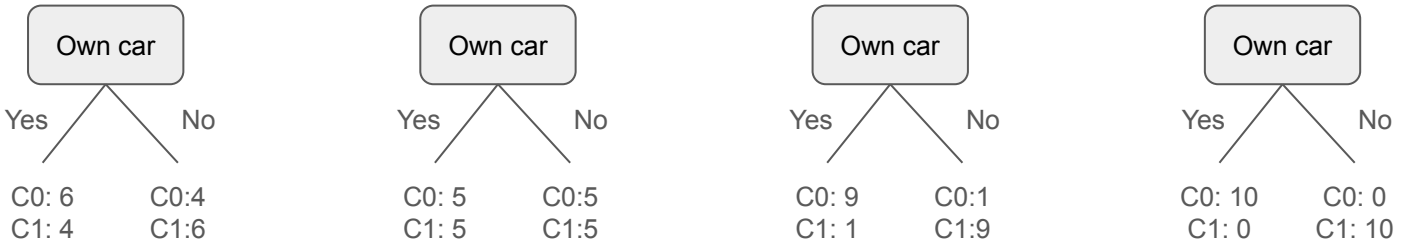
Before split
Class 0 (C0) = 10 records
Class 1 (C1) = 10 records







Best Split: Node Impurity



- Ideally, the group should be as pure as possible (aka homogeneous)
- In real data, it is challenging to find pure segments

Node Impurity: Entropy

- Entropy is a measure of disorder
- Disorder → how mixed (impure) the segment is, with respect to the target
- Defined as:

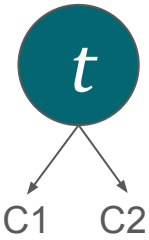
$$Entropy(t) = -\sum_j p(j|t) \log_2 p(j|t)$$

Where $p(j|t)$ is the relative frequency of class j at node t

High Entropy → High Impurity → Heterogenous → **Discouraged**

Low Entropy → Low Impurity → Homogeneous → **Encouraged**

Entropy: Example



$$Entropy(t) = -\sum_j p(j|t) \log_2 p(j|t)$$

Class

C1	0
C2	6

$p(C1|t) = 0/6 = 0$
$p(C2|t) = 6/6 = 1$
$Entropy(t) = -0 \log_2 0 - 1 \log_2 1 = 0$

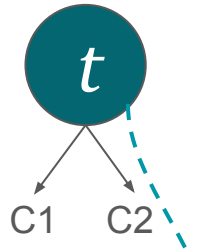
C1	1
C2	5

$p(C1|t) = 1/6$
$p(C2|t) = 5/6$
$Entropy(t) = -(1/6) \log_2 (1/6) - (5/6) \log_2 (5/6) = 0.65$

C1	2
C2	4

$p(C1|t) = 2/6$
$p(C2|t) = 4/6$
$Entropy(t) = -(2/6) \log_2 (2/6) - (4/6) \log_2 (4/6) = 0.92$

Entropy: Example (Cont.)



C1=Yes	0
C2=No	6

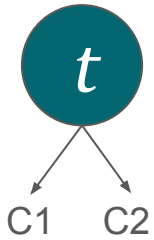
buys_computer
No
No
No
No
No
No

C1=Yes	1
C2=No	5

buys_computer
No
No
No
Yes
No
No

C1=Yes	2
C2=No	4

buys_computer
Yes
No
No
Yes
No
No



Is this enough?

Ref: https://tenor.com/search/not-enough-gifs

Attribute Selection: Information Gain

- How informative an attribute is, with respect to the target
- Informative Gain measures the change in entropy due to further splitting
- Defined as:

$$Gain_{split} = Entropy(p) - \left(\sum_{i=1}^{k} \frac{n_i}{n} Entropy(i) \right)$$

Where parent node p is split into k partitions; n_i is number of records in partition i

High IG_{split} → High gain → High reduction in entropy → **Encouraged**

Low IG_{split} → Low gain → Low reduction in entropy → **Discouraged**

Information Gain: Example

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

Want to predict (Target)

$$Entropy(t) = -\sum_j p(j|t)\log_2 p(j|t)$$

$$Gain_{split} = Entropy(p) - \left(\sum_{i=1}^{k} \frac{n_i}{n} Entropy(i)\right)$$

At a glance…

- Compute entropy based on the target attribute t
- For each partition i in k remaining partitions:
  - Compute entropy based on partition i
  - Apply the weighted ratio
- Compute summation of all k weighted entropies
- Compute the final information gain if splitting upon **[selected attribute]**

$$Entropy(t) = -\sum_j p(j|t)\log_2 p(j|t)$$

$$Gain_{split} = Entropy(p) - \left(\sum_{i=1}^{k} \frac{n_i}{n} Entropy(i)\right)$$

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

Want to predict (Target)

At a glance…

- Compute entropy based on the target attribute t
- For each partition i in k remaining partitions:
  - Compute entropy based on partition i
  - Apply the weighted ratio
- Compute summation of all k weighted entropies
- Compute the final information gain if splitting upon **age**

$$Entropy(t) = -\sum_j p(j|t)\log_2 p(j|t)$$

$$p(yes|t) = 9/14$$
$$p(no|t) = 5/14$$

$$\therefore Entropy(t)$$
$$= -\frac{9}{14}\log_2\left(\frac{9}{14}\right) - \frac{5}{14}\log_2\left(\frac{5}{14}\right)$$
$$= 0.940$$

At a glance…

$$Entropy(t) = -\sum_j p(j|t)\log_2 p(j|t) = 0.940$$

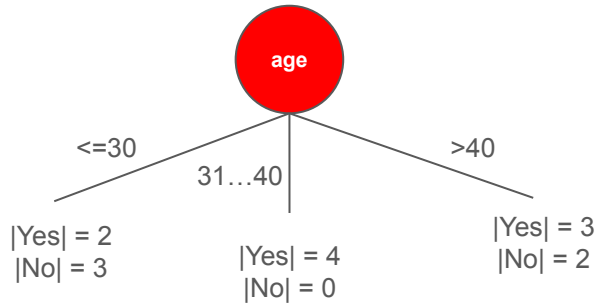
$$Gain_{split} = Entropy(p) - \left(\sum_{i=1}^{k} \square\, Entropy(i)\right)$$

- Compute entropy based on the target attribute t
- For each partition i in k remaining partitions:
  - Compute entropy based on partition i
  - Apply the weighted ratio
- Compute summation of all k weighted entropies
- Compute the final information gain if splitting upon **age**

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

Want to predict (Target)

$$Entropy(age) = -\sum_j p(j|age)\log_2 p(j|age)$$



Let's turn this ⬆️ into table format

$$Entropy(age) = -\sum_j p(j|age)\log_2 p(j|age)$$

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

Want to predict (Target)

age	j=yes	j=no	log(j|age)
<=30	2	3	0.971
31…40	4	0	
>40	3	2	

$$p(yes|\leq 30) = 2/5$$
$$p(no|\leq 30) = 3/5$$

$$\therefore Entropy(age)$$
$$= -\frac{2}{5}\log_2\left(\frac{2}{5}\right) - \frac{3}{5}\log_2\left(\frac{3}{5}\right)$$
$$= 0.971$$

At a glance…

$$Entropy(t) = -\sum_j p(j|t)\log_2 p(j|t) = 0.940$$

$$Gain_{split} = Entropy(p) - \left(\sum_{i=1}^{k} \frac{n_i}{n} Entropy(i)\right)$$

- Compute entropy based on the target attribute t
- For each partition i in k remaining partitions:
  - Compute entropy based on partition i
  - Apply the weighted ratio
- Compute summation of all k weighted entropies
- Compute the final information gain if splitting upon **age**

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

Want to predict (Target)

age	j=yes	j=no	log(j|age)	n_i
<=30	2	3	0.971	5
31…40	4	0		
>40	3	2		

$$n_i = 5$$
$$n = 14$$
$$\therefore \frac{n_i}{n} Entropy(\leq 30) = \frac{5}{14}(0.971)$$
$$= 0.347$$

At a glance…

$$Entropy(t) = -\sum_j p(j|t)\log_2 p(j|t) = 0.940$$

$$Gain_{split} = Entropy(p) - \left(\sum_{i=1}^{k} \frac{n_i}{n} Entropy(i)\right)$$

- Compute entropy based on the target attribute t
- For each partition i in k remaining partitions:
  - Compute entropy based on partition i
  - Apply the weighted ratio
- Compute summation of all k weighted entropies
- Compute the final information gain if splitting upon **age**

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

Want to predict (Target)

age	j=yes	j=no	log(j|age)	n_i
<=30	2	3	0.971	5
31…40	4	0	0	4
>40	3	2	0.971	5

$$= \frac{5}{14}(0.971) + \frac{4}{14}(0) + \frac{5}{14}(0.971)$$
$$= 0.694$$

At a glance…

$$Entropy(t) = -\sum_j p(j|t)\log_2 p(j|t) = 0.940$$

$$Gain_{split} = Entropy(p) - \left(\sum_{i=1}^{k} \frac{n_i}{n} Entropy(i)\right)$$

- Compute entropy based on the target attribute t
- For each partition i in k remaining partitions:
  - Compute entropy based on partition i
  - Apply the weighted ratio
- Compute summation of all k weighted entropies
- Compute the final information gain if splitting upon **age**

age	income	student	credit_rating	buys_computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

Want to predict (Target)

$$Gain_{split}(age) = 0.940 - 0.694 = 0.246$$

Activity

Now, *as a group*, work on these remaining attributes:

$$Gain_{split}\,(income) = ?$$

$$Gain_{split}\,(student) = ?$$

$$Gain_{split}\,(credit\,rating) = ?$$

age	income	student	credit rating	buys computer
<=30	high	no	fair	no
<=30	high	no	excellent	no
31…40	high	no	fair	yes
>40	medium	no	fair	yes
>40	low	yes	fair	yes
>40	low	yes	excellent	no
31…40	low	yes	excellent	yes
<=30	medium	no	fair	no
<=30	low	yes	fair	yes
>40	medium	yes	fair	yes
<=30	medium	yes	excellent	yes
31…40	medium	no	excellent	yes
31…40	high	yes	fair	yes
>40	medium	no	excellent	no

How does this become a tree?

Ref: https://makeameme.org/meme/cute-sea-lion

Decision Tree Algorithm

Algorithm 26.1: Decision Tree Algorithm

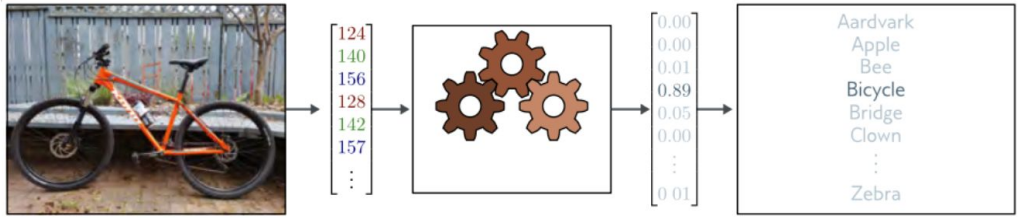
```
DecisionTree (D):
if (stop condition met) then
    Choose majority class in D as the leaf label
foreach (attribute A in D) do
    if (A is numeric) then
        Try all possible distinct splits: A ≤ v
    if (A is categorical) then
        Try all possible distinct splits: A ∈ V
Choose the best split based on highest information gain
Partition D into D_L and D_R based on the best split
DecisionTree(D_L)
DecisionTree(D_R)
```

Uh-oh..

Ref: https://www.mihaileric.com/posts/decision-trees/

Recall this? Classification Steps

Image recognition
(Classification)



Feature Extraction	Choose relevant features that contribute most to the classification task
Data Splitting	Divide the dataset into training data and testing data
Model Training	Choose a classification algorithm and train it on the training data
Hyperparameter Tuning	Fine-tune the model by adjusting hyperparameters to optimize its performance
Model Evaluation	Evaluate the model performance on the testing data using evaluation metrics
Classification/Prediction	Make predictions on new unseen data

Training & Testing in Decision Tree

Steps in Training & Testing:

1. *Split the Data* → Divide dataset into training (80%) and testing (20%).
2. *Train the Model* → Fit a Decision Tree using the training data.
3. *Test the Model* → Use the trained model to predict on test data.
4. *Evaluate Performance* → Compare predictions with actual values (e.g., Accuracy Score).

More to Consider

- Too many leaf nodes?
- Generalize enough?
- Parameters?
- Got the results, but how to interpret?
- And more…

Homework

- Study **Gini Index**
- Compute the toy example using Gini Index
- Change criterion in the imported library, using Gini Index
- Compare Gini Index vs Entropy
- Use another dataset (dataset.csv)
- Play with parameters:
  - max_depth
  - min_samples_split
  - min_samples_leaf
- Explain your understanding after trying these different parameters

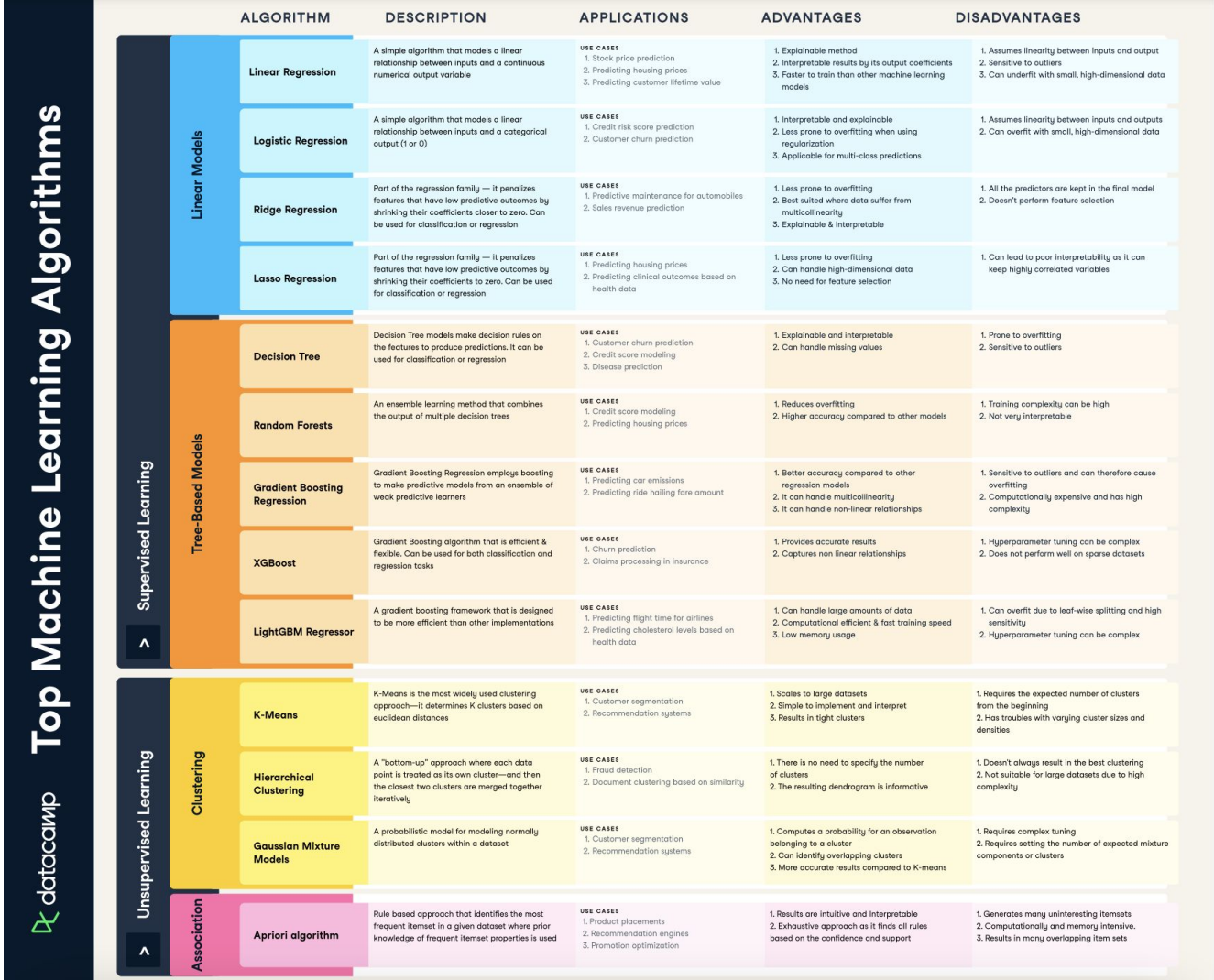
In Summary

- Introducing Model Concept
- Introducing Machine Learning
  - Algorithms
  - Types
- Classification Concept
- Decision Tree

Ref: https://9gag.com/gag/52045

Q & A

Resources


datacamp
Top Machine Learning Algorithms
ALGORITHM
DESCRIPTION
APPLICATIONS
ADVANTAGES
DISADVANTAGES
Supervised Learning
Linear Models
Linear Regression
A simple algorithm that models a linear relationship between inputs and a continuous numerical output variable
USE CASES
1. Stock price prediction
2. Predicting housing prices
3. Predicting customer lifetime value
1. Explainable method
2. Interpretable results by its output coefficients
3. Faster to train than other machine learning models
1. Assumes linearity between inputs and output
2. Sensitive to outliers
3. Can underfit with small, high-dimensional data
Logistic Regression
A simple algorithm that models a linear relationship between inputs and a categorical output (1 or 0)
USE CASES
1. Credit risk score prediction
2. Customer churn prediction
1. Interpretable and explainable
2. Less prone to overfitting when using regularization
3. Applicable for multi-class predictions
1. Assumes linearity between inputs and outputs
2. Can overfit with small, high-dimensional data
Ridge Regression
Part of the regression family — it penalizes features that have low predictive outcomes by shrinking their coefficients closer to zero. Can be used for classification or regression
USE CASES
1. Predictive maintenance for automobiles
2. Sales revenue prediction
1. Less prone to overfitting
2. Best suited where data suffer from multicollinearity
3. Explainable & interpretable
1. All the predictors are kept in the final model
2. Doesn't perform feature selection
Lasso Regression
Part of the regression family — it penalizes features that have low predictive outcomes by shrinking their coefficients to zero. Can be used for classification or regression
USE CASES
1. Predicting housing prices
2. Predicting clinical outcomes based on health data
1. Less prone to overfitting
2. Can handle high-dimensional data
3. No need for feature selection
1. Can lead to poor interpretability as it can keep highly correlated variables
Tree-Based Models
Decision Tree
Decision Tree models make decision rules on the features to produce predictions. It can be used for classification or regression
USE CASES
1. Customer churn prediction
2. Credit score modeling
3. Disease prediction
1. Explainable and interpretable
2. Can handle missing values
1. Prone to overfitting
2. Sensitive to outliers
Random Forests
An ensemble learning method that combines the output of multiple decision trees
USE CASES
1. Credit score modeling
2. Predicting housing prices
1. Reduces overfitting
2. Higher accuracy compared to other models
1. Training complexity can be high
2. Not very interpretable
Gradient Boosting Regression
Gradient Boosting Regression employs boosting to make predictive models from an ensemble of weak predictive learners
USE CASES
1. Predicting car emissions
2. Predicting ride hailing fare amount
1. Better accuracy compared to other regression models
2. It can handle multicollinearity
3. It can handle non-linear relationships
1. Sensitive to outliers and can therefore cause overfitting
2. Computationally expensive and has high complexity
XGBoost
Gradient Boosting algorithm that is efficient & flexible. Can be used for both classification and regression tasks
USE CASES
1. Churn prediction
2. Claims processing in insurance
1. Provides accurate results
2. Captures non linear relationships
1. Hyperparameter tuning can be complex
2. Does not perform well on sparse datasets
LightGBM Regressor
A gradient boosting framework that is designed to be more efficient than other implementations
USE CASES
1. Predicting flight time for airlines
2. Predicting cholesterol levels based on health data
1. Can handle large amounts of data
2. Computational efficient & fast training speed
3. Low memory usage
1. Can overfit due to leaf-wise splitting and high sensitivity
2. Hyperparameter tuning can be complex
Unsupervised Learning
Clustering
K-Means
K-Means is the most widely used clustering approach—it determines K clusters based on euclidean distances
USE CASES
1. Customer segmentation
2. Recommendation systems
1. Scales to large datasets
2. Simple to implement and interpret
3. Results in tight clusters
1. Requires the expected number of clusters from the beginning
2. Has troubles with varying cluster sizes and densities
Hierarchical Clustering
A "bottom-up" approach where each data point is treated as its own cluster—and then the closest two clusters are merged together iteratively
USE CASES
1. Fraud detection
2. Document clustering based on similarity
1. There is no need to specify the number of clusters
2. The resulting dendrogram is informative
1. Doesn't always result in the best clustering
2. Not suitable for large datasets due to high complexity
Gaussian Mixture Models
A probabilistic model for modeling normally distributed clusters within a dataset
USE CASES
1. Customer segmentation
2. Recommendation systems
1. Computes a probability for an observation belonging to a cluster
2. Can identify overlapping clusters
3. More accurate results compared to K-means
1. Requires complex tuning
2. Requires setting the number of expected mixture components or clusters
Association
Apriori algorithm
Rule based approach that identifies the most frequent itemset in a given dataset where prior knowledge of frequent itemset properties is used
USE CASES
1. Product placements
2. Recommendation engines
3. Promotion optimization
1. Results are intuitive and interpretable
2. Exhaustive approach as it finds all rules based on the confidence and support
1. Generates many uninteresting itemsets
2. Computationally and memory intensive.
3. Results in many overlapping item sets